pattern known as *double submit cookies*. This is what Razor Pages uses by default, and this is how it works: A random value is generated by the server and sent to the browser in a cookie. The cookie has its `SameSite` attribute set to `Strict`, ensuring it can be sent by the browser only if the request originates from the site that issued the cookie. The value, the *request verification token*, is also included as a request parameter in a hidden form field. When a form is submitted, the framework checks the value in the hidden field against that of the cookie. If they are both present and match, the request is verified as having originated from the original website and not a third party. Using this technique, it is impossible for spoofsite.com to send the request verification cookie to mysavings.com.

As you learned in chapter 6, the hidden form field containing the request verification token is generated automatically by the form tag helper whenever you set the method of a form element to `post`. You also learned how to generate the token directly, using the `IAntiforgery` service, or as a hidden form field, using the `Html.AntiForgeryToken` helper, in the event you aren't using a form, in chapter 11, when we looked at working with client-side code.

CONFIGURING ANTIFORGERY OPTIONS

Various options relating to the antiforgery feature are configurable via the `AntiforgeryOptions` class, as shown in table 13.1.

Table 13.1 The `AntiforgeryOptions`